

Московский авиационный институт (национальный исследовательский университет)

Институт «Компьютерные науки и прикладная математика»

Кафедра «Вычислительная математика и программирование»

Лабораторная работа №5

по курсу «Дискретный анализ»

на тему «Суффиксные деревья»

Студент: Чигрин Никита Александрович

Группа: М8О-208БВ-24

Преподаватель: Макаров Н.К.

Проверил: Сахарин Н.А.

Дата: 16 мая 2026

Оценка: _____

Подпись: _____

Москва, 2026

1 Постановка задачи

Найти все вхождения заданного образца P (pattern) в тексте T (text) с использованием статистики совпадений. Для решения требуется применить структуру данных «суффиксное дерево», построенное для текста T , после чего выполнить поиск образца P за время, пропорциональное длине образца.

Более формально: Пусть $T = t_1 t_2 \dots t_n$ — исходный текст, $P = p_1 p_2 \dots p_m$ — образец. Требуется найти все позиции i такие, что $T[i : i + m - 1] = P$.

2 Формат ввода и вывода

Входные данные

Строка 1: образец P (строка, состоящая из строчных латинских букв)

Строка 2: текст T (строка, состоящая из строчных латинских букв)

Выходные данные

Целые числа (по одному на строку) — позиции (индексация с 1) в тексте T , с которых начинается вхождение образца P . Позиции выводятся в порядке возрастания.

Ограничения

- $1 \leq |P| \leq 10^5$
- $1 \leq |T| \leq 10^5$
- Суммарное время работы: $O(|T| + |P|)$
- Память: $O(|T|)$

3 Метод решения (алгоритм)

Используется **алгоритм Укконена** для построения суффиксного дерева за линейное время. Суффиксное дерево — это компактное префиксное дерево (бор), содержащее все суффиксы строки T . Каждое ребро дерева помечено подстрокой текста, а листья соответствуют суффиксам.

3.1 Основные понятия

- **Суффикс** строки T — это подстрока, заканчивающаяся в конце T : $T[i..n]$ для $i = 1, 2, \dots, n$.

- **Суффиксное дерево** — дерево, в котором:
 1. Каждому суффиксу соответствует путь от корня до листа.
 2. Внутренние вершины имеют не менее двух детей.
 3. Рёбра помечены нетривиальными подстроками.
 4. Никакие два ребра из одной вершины не начинаются с одного символа.
- **Суффиксная ссылка** — указатель из вершины, соответствующей строке $x\alpha$ (где x — символ), в вершину, соответствующую строке α . Используется для эффективного построения.

3.2 Алгоритм Укконена (идея)

Алгоритм строит дерево итеративно, добавляя символы текста один за другим. На i -й итерации в дерево добавляются все суффиксы строки $T[1..i]$. Благодаря суффиксным ссылкам и специальным правилам (правило 1, правило 2, правило 3) достигается линейное время.

Три правила продления

Пусть мы находимся в точке, соответствующей суффиксу $T[j..i-1]$, и хотим добавить символ $T[i]$.

1. **Правило 1 (лист):** если текущая позиция — лист, добавляем символ в метку ребра.
2. **Правило 2 (ветвление):** если нет ребра с символом $T[i]$, создаём новое ребро.
3. **Правило 3 (останов):** если ребро с $T[i]$ существует, ничего не делаем и завершаем текущую фазу.

3.3 Поиск образца

После построения дерева для T поиск P выполняется так:

1. Начинаем из корня.
2. Для очередного символа $P[k]$ идём по ребру, начинающемуся с этого символа.
3. Если на каком-то шаге нужного ребра нет — образец отсутствует.
4. Если все символы P прочитаны, то в поддереве текущей вершины находятся все вхождения P (все листья этого поддерева дают начальные позиции суффиксов, содержащих P).

Время поиска $O(|P| + \text{ответ})$.

3.4 Сбор статистики совпадений

Чтобы получить все позиции, достаточно обойти поддерево вершины, в которой закончился образец, и собрать номера листьев. Каждый лист хранит начальную позицию соответствующего суффикса.

4 Сложность

- Построение дерева: $O(|T|)$ времени и памяти.
- Поиск образца: $O(|P|)$ без учёта вывода.
- Вывод позиций: $O(\text{число вхождений})$.

5 Тестирование

Пример 1 (базовый)

Входные данные:

```
aba  
abababa
```

Выходные данные (позиции с 1):

```
1  
3  
5
```

Пояснение: образец "aba" встречается в позициях 1, 3, 5 строки "abababa".

Пример 2 (из вашего файла)

Входные данные:

```
aba  
qababaz
```

Выходные данные:

```
2  
4
```

Пояснение: Текст = "q a b a b a z": - Позиция 2: "aba" (символы 2–4) - Позиция 4: "aba" (символы 4–6)

Пример 3 (образец длиннее текста)

Входные данные:

abcdef

abc

Выходные данные: (пусто)

Пример 4 (повторяющиеся символы)

Входные данные:

aaa

aaaaa

Выходные данные:

1

2

3

Пояснение: "aaa" входит в позициях 1, 2, 3 строки "aaaaa".

6 Вывод

В ходе выполнения лабораторной работы был реализован алгоритм Укконена для построения суффиксного дерева и решена задача поиска всех вхождений образца в текст с использованием этой структуры.

Сравнение с другими подходами:

- **Z-функция** даёт $O(|T| + |P|)$ времени, простой в реализации, но требует $O(|T|)$ дополнительной памяти и плохо обобщается на множественные запросы.
- **Суффиксное дерево** требует больше кода и константных затрат, однако при множественных запросах к одному и тому же тексту (k запросов) время составит $O(|T| + k \cdot |P|)$, что значительно быстрее перестроения Z-функции для каждого запроса.

Таким образом, суффиксные деревья особенно эффективны в задачах, где текст фиксирован, а образцы разнообразны и поступают в большом количестве. Для единичного поиска проще и быстрее использовать Z-функцию или алгоритм Кнута–Морриса–Пратта, но в рамках данной работы было важно освоить построение суффиксного дерева как фундаментальной структуры для сложных текстовых алгоритмов.